EE4207 Web Application Development

Mini Project Report

PortfolioCraft

A MERN-Stack Portfolio Management Platform with a Visual CMS Editor

Name	Priyanath R.S.M.
Index number	EG/2023/5800
Submission date	2026.10.31
GitHub repository	https://github.com/rsmpdo/portfolio-platform
Live deployment	https://portfoliocraftlk.vercel.app

Abstract

PortfolioCraft lets creators build, customise and publish professional portfolio websites without writing code. A creator assembles a portfolio from reusable section components in a three-pane visual editor, themes it, previews it at desktop, tablet and mobile widths and publishes it at a public address of the form /p/ handle. The system is a MERN application: a React 18 single-page client, a stateless Express REST API secured with JWT, and MongoDB for persistence, with Cloudinary for media and e-mail delivery for account and billing notifications. The main entity, the portfolio Layout, supports full Create, Read, Update and Delete operations, and five further collections (users, messages, payment requests, job applications and newsletter subscribers) are managed through forms, filtered tables and an administrator dashboard. This report documents the requirements, architecture, database design, CRUD implementation, validation and security measures, user interface, code organisation and deployment of the system.

Report map against the evaluation criteria

Criteria	**Report section**
Create, Read, Update, Delete	CRUD Implementation
Validation and Error Handling	Validation and Security
Database Design	Database Design
UI/UX and Responsiveness	User Interface
Code Structure	Code Organization and Structure
Documentation	Under documentation/ in repository
Demonstration	Separate video
Limitations, conclusion	Future Work

Introduction

A portfolio is the main way developers, designers and other creators present their work to employers and clients, yet building one usually means either hand-coding a site or accepting the rigid layouts of a template builder. PortfolioCraft addresses this by treating a portfolio as data: an ordered list of typed section components (hero, about, projects, experience, skills and so on) stored as a JSON document in MongoDB and rendered on demand by a registry of React components. Because the layout is data, a creator can reorder sections, edit their content, switch themes or whole templates, and publish the result instantly.

Objectives

The project is set out to:

- provide a secure account system (registration with e-mail verification, JWT sessions, optional two-factor authentication);
- implement complete, persistent CRUD on the portfolio Layout entity and on the supporting entities, with safeguards on destructive actions;
- deliver a visual editor with live preview, undo/redo and device-width simulation, and a public rendering route for published portfolios;
- give administrators moderation tools (user plans, portfolio ban/publish, payment review, messages and job applications).

Users and roles

Three kinds of user interact with the system: anonymous visitors who view public portfolios and submit contact messages, newsletter sign-ups or job applications; registered creators (role: standard) on a Free, Pro or Studio plan who may own 1, 3 or 5 portfolios respectively; and administrators (role: admin) who access the moderation dashboard. The portfolio Layout is the main entity for the CRUD requirement.

System Overview

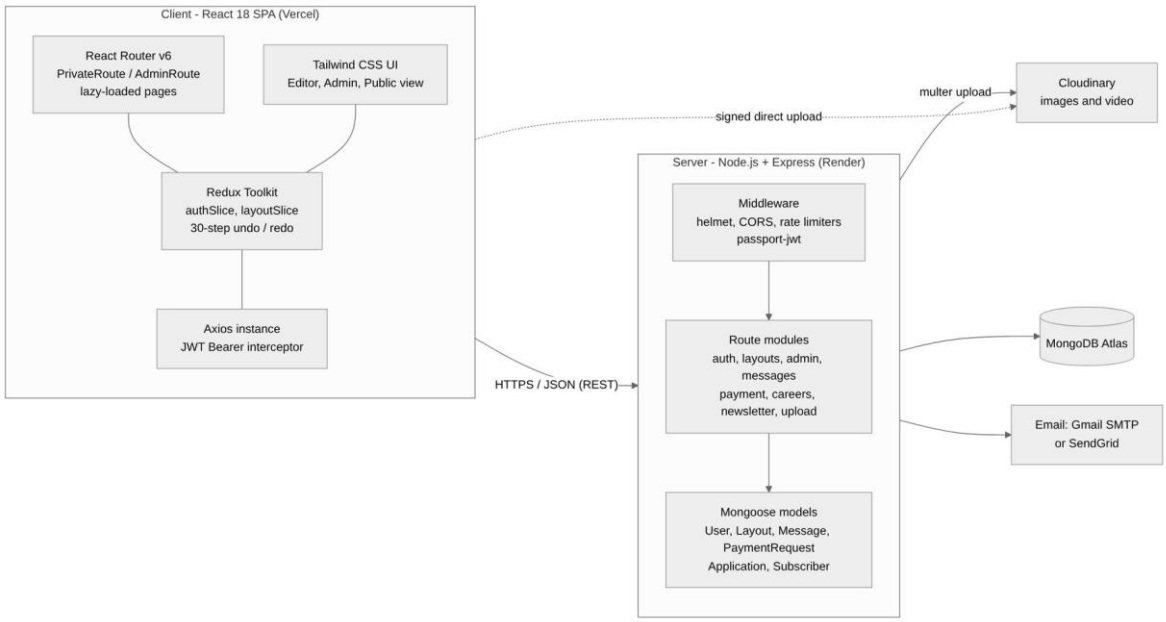
The platform is organised into functional modules. Each is implemented as a pair of React pages or components and an Express route module.

Module	Capabilities (as implemented in the repository)
Accounts and security	Registration, e-mail verification (24 h hashed token), login, password reset (10 min token), profile and password update, account deletion, TOTP two-factor authentication with recovery codes.
Visual editor	Three panes (sections list, live canvas, property inspector); add, reorder, hide and delete sections; theme colour tokens and font; template switching; draft/publish; 30-step undo/redo (Ctrl+Z / Ctrl+Y); desktop, tablet and mobile preview.
Section library	12 section types in the editor: Hero Banner, About Me, Projects Grid, Experience Timeline, Skills Cloud, Testimonials, Social Links, Contact, Media Gallery, Case Study (Markdown), Interactive Embed (Pro) and Data Metrics (Studio).
Public portfolio	/p/:handle loads a published, non-banned layout, applies the theme through CSS variables, records page views and referrer source, and offers a one-click ATS-friendly resume print view.
Plans and billing	Free, Pro ($12) and Studio ($29) one-time plans. Users submit a bank-transfer slip and ID document; an administrator approves or rejects and approval upgrades the plan and sends an e-mail.
Messaging	Visitors send enquiries from a portfolio contact form; creators read, mark as read and delete them; administrators see privacy-preserving metadata.
Administration	Dashboard with user table (search, plan filter, plan change, warn, delete), portfolio moderation (publish/ban/delete), payment review, message log and job applications.
Marketing site	Home, Features, Templates, Showcase, Pricing, Blog (newsletter), Careers (applications), Contact, legal pages.

Architecture and Technology Stack

PortfolioCraft is a decoupled three-tier application. The React client talks to the Express API exclusively through JSON over HTTPS; the API authenticates each protected request with a Bearer JWT, applies role and ownership checks, and reads or writes MongoDB through Mongoose models. Images and video are

stored on Cloudinary, uploaded either through the API (multer) or directly from the browser using a server-signed request. Transactional e-mail is sent through Gmail SMTP or SendGrid.



System architecture and deployment targets

Layer	Technologies (versions from package.json)	Role
Client	React 18.2, Vite 5, React Router 6.20, Redux Toolkit 1.9, Axios 1.6	SPA, routing with route guards, state and HTTP
UI	Tailwind CSS 3.3, Framer Motion 10, Lucide icons, Google Fonts	Styling, animation, icons, 9 selectable fonts
API	Node.js 18, Express 4.18, Passport 0.6 with passport-jwt 4, helmet 7, express-rate-limit 6, cors	REST routes, authentication, hardening
Data	MongoDB (Atlas), Mongoose 7.5	Persistence, schema validation, indexes
Services	bcryptjs, jsonwebtoken, Node crypto (TOTP), Cloudinary + multer, Nodemailer, SendGrid	Hashing, tokens, 2FA, media and e-mail
DevOps	GitHub Actions, Vercel (client), Render (API), render.yaml, vercel.json	CI build checks and hosting

Typical request : Opening /p/alex makes the client call GET /api/layouts/public/alex. The API finds the layout where handle matches, isPublished is true and isBanned is not true, populates the owner's public profile fields, replies immediately, and updates the view counters asynchronously so that analytics never delay the response. The client then maps each component object to its React component through a registry (ComponentRenderer), passing the stored props.

State management : Two Redux slices hold the application state: authSlice (user, token persisted in localStorage, 2FA challenge) and layoutSlice (active layout, the creator's portfolios, public layout, and past/future history stacks). Every editing reducer first pushes a deep-copied snapshot onto past (capped at 30), which gives undo and redo without any server round-trip; changes reach MongoDB only when the creator saves or publishes.

Database Design

MongoDB was chosen because a portfolio is a hierarchical, variable-shaped document: two creators may have entirely different sections, each with different properties. The sections are therefore embedded in the Layout document as an array (components), each with a props field of type Mixed, while relationships between independent entities (a user and their layouts, messages and payment requests) use referenced ObjectIds. Figure 2 shows the resulting entity-relationship diagram, derived from the Mongoose models in backend/models/.



Entity-relationship diagram of the related collections (PK = primary key, FK = foreign key reference, UK = unique index). Application, Subscriber and the legacy Payment collection are listed in the table below.

Collection	Keys and indexes	Typed fields and constraints
User	PK _id; unique username, email	username String 3–30; email lower-case, regex-validated; password min 6, bcrypt, select:false; role enum standard/admin; plan enum free/pro/studio; Boolean flags; reset/verification tokens with Date expiry; profile fields with maxlength.
Layout (main)	PK _id; FK userId→User (indexed); unique handle; compound indexes {handle,isPublished,isBanned} and {userId,updatedAt}	handle lower-case slug; isPublished, isBanned Boolean; theme (colour tokens, font, templateId); seo; analytics (counters, 30-day dailyViews[], referrers[]); components[] with type enum (13 values), order Number, isVisible, props Mixed.
Message	PK _id; FK recipientUserId→User (nullable)	senderName, senderEmail, message required; subject; recipientHandle; isRead Boolean; timestamps.
PaymentRequest	PK _id; FKs userId, reviewedBy→User; index {status,createdAt}	plan enum pro/studio; amount Number; fullName, nationalIdNumber, country, phone, paymentSlipUrl, nationalIdUrl required; status enum pending/approved/rejected; adminNote; reviewedAt.
Application	PK _id; unique referenceId	jobId, jobTitle, fullName, email, portfolioUrl, resumeUrl required; status enum pending/reviewed/shortlisted/rejected.
Subscriber	PK _id; unique email	email regex-validated; isActive Boolean; subscribedAt Date.
Payment	PK _id; FK userId; unique transactionId	Legacy card-payment record; only read by the admin statistics endpoint.

Integrity : Uniqueness is enforced by database indexes, enumerations by schema validators, and secrets (password, twoFactorSecret, twoFactorTempSecret, twoFactorRecoveryCodes) are excluded from queries by default with select:false. MongoDB has no foreign-key cascade, so the cascade is implemented in application code: deleting an account removes the user's layouts and received messages before the user document itself.

CRUD Implementation

All write operations on private data are protected by passport.authenticate("jwt"), and creator-facing queries are scoped to the signed-in user ({ _id, userId: req.user._id }), so a creator can never read or modify another creator's layout. Table 1 lists the operations on the main entity, the portfolio Layout.

Operation	Endpoint	Behaviour, user interface and safeguards
Create	POST /api/layouts/new; POST /api/auth/register	Creates a layout from a form (title, handle, template) in *Profile → My Portfolios*. The API counts the user's layouts against the plan limit (Free 1, Pro 3, Studio 5) and returns 403 when exceeded and returns 400 if the handle is taken. Registration also seeds a starter layout with four sections. New sections are added in the editor's *Add Section* list.
Read	GET /api/layouts/my-portfolios; GET /api/layouts/me?id=; GET /api/layouts/public/:handle; GET /api/admin/layouts	Lists a creator's portfolios with plan limit; loads one for editing; serves a published portfolio publicly (404 if unpublished or banned). Admin table supports text search (title, handle, owner) and status filters (published, unpublished, banned, system). The Templates page filters by tier (All, Free, Pro, Studio).
Update	PUT /api/layouts/me; admin PUT /api/admin/layout/:id/toggle-publish and /toggle-ban	The editor saves title, handle, theme, SEO, components and publish state in one request, merged into the stored document. Inspector edits, reordering (move up/down), visibility toggles, theme tokens and template switching all update the Redux state first and are saved with *Save* or *Publish*.
Delete	DELETE /api/layouts/id/:layoutId; DELETE /api/layouts/component/:id; admin DELETE /api/admin/layout/:id	A confirmation modal names the portfolio. The API refuses to delete a creator's only portfolio (400) and deletes only if the user owns it (404 otherwise). Removing a section in the editor is undoable. Admin deletion requires re-entering the admin password.

The two safeguards that matter most, the plan limit on create and the last-portfolio guard on delete, are implemented server-side so they cannot be bypassed from the client:

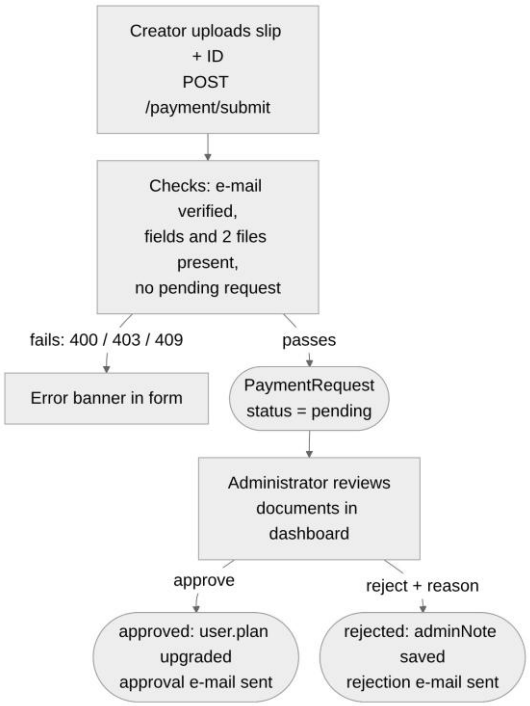
```
// routes/layout.js  (excerpt)
const currentCount = await Layout.countDocuments({ userId: req.user._id });
if (currentCount >= getPlanLimit(req.user))
  return res.status(403).json({ success: false, message: 'Portfolio limit reached ...' });

const count = await Layout.countDocuments({ userId: req.user._id });
if (count <= 1) return res.status(400).json({ success: false, message: 'Cannot delete your only portfolio' });
const layout = await Layout.findOne({ _id: req.params.layoutId, userId: req.user._id });
if (!layout) return res.status(404).json({ success: false, message: 'Portfolio layout not found' });
await layout.deleteOne();
```

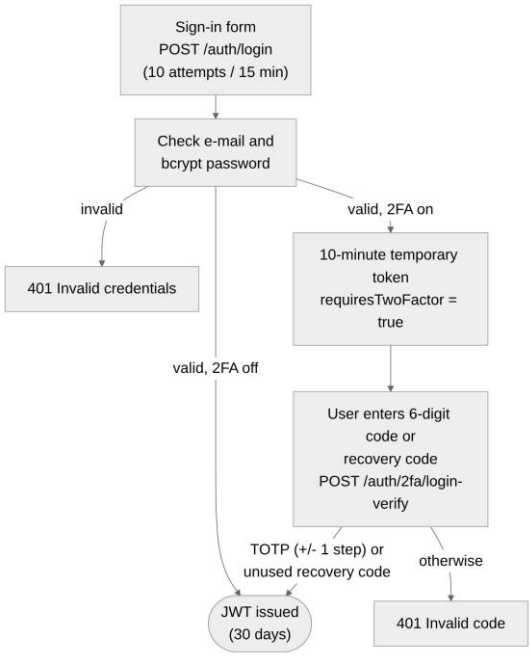
The supporting entities follow the same pattern.

Entity	Create	Read	Update	Delete
User	POST /auth/register (form with confirm-password)	GET /auth/me; admin GET /admin/users (search, plan filter)	PUT /auth/profile, PUT /auth/password; admin PUT /admin/user/:id/plan	POST /auth/delete-account (password, cascades); admin DELETE /admin/user/:id (admin password)
Message	POST /messages (public contact form)	GET /messages/me (inbox, unread count); admin GET /messages/admin (metadata)	GET /messages/:id/read toggles read state	DELETE /messages/:id (owner or admin, confirmation modal)
Payment request	POST /payment/submit (multipart: slip + ID)	GET /payment/my-requests; admin GET /admin/payment-requests (status, plan, text filter)	Admin PUT .../approve or .../reject (plan upgrade + e-mail)	Retained for audit (no delete route)
Application	POST /careers/apply	Admin GET /careers/admin (search, status tabs)	Admin PUT /careers/admin/:id/status	Admin DELETE /careers/admin/:id (confirmation)
Subscriber	POST /newsletter/subscribe	GET /newsletter/count	Re-subscribing re-activates isActive	Not implemented

Two multi-step workflows show how these rules combine : the plan-upgrade approval process, which moves a payment request through its pending, approved and rejected states, and the two-step sign-in used by accounts with two-factor authentication. Only requests in the pending state can be reviewed; any other state returns 400.



(a) Plan upgrade: submission and admin review



(b) Sign-in with optional two-factor step

Key workflows implemented in the API

Validation, Error Handling and Security

Client-side validation : Forms use HTML5 constraints (required, type="email", minLength=6) plus script checks: the registration form rejects mismatched passwords and a missing admin code before any request is made; the payment form requires both documents; submit buttons show a spinner and are disabled while a request is in flight; and errors appear in inline alert banners rather than browser dialogs.

Server-side validation : Every route re-validates its input and returns a JSON body { success, message } with a meaningful status code: 400 for missing or invalid fields and duplicates, 401 for a bad or missing token or password, 403 for role, plan-limit or unverified-email violations, 404 for unknown resources, 409 for a duplicate pending payment request, and 500 with the error message for unexpected failures. Mongoose adds type, enum, length and regex validation, and unique indexes reject duplicates. A global 404 handler and a global error handler ensure the API always answers in JSON, and a failed database connection is logged without crashing the process.

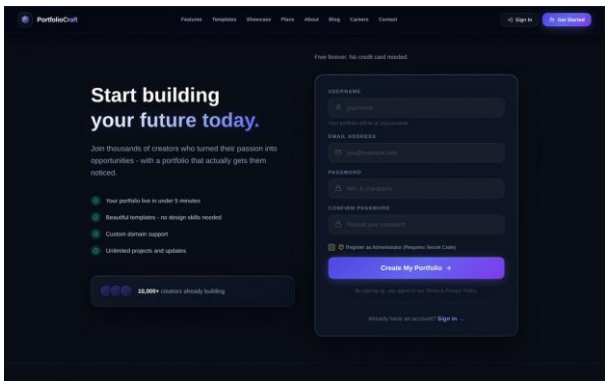
Authentication and authorization

The main protections are:

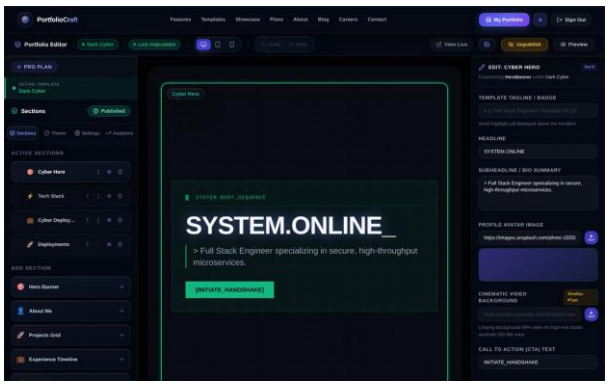
- passwords hashed with bcrypt (10 salt rounds); verification and reset tokens are random 32-byte values stored only as SHA-256 hashes with an expiry; login errors do not reveal whether the e-mail exists;
- JWT Bearer tokens (30-day default) verified by passport-jwt; an AdminRoute guard on the client and a checkAdmin middleware on the server; unverified accounts see a verification page instead of the editor and cannot submit payments;
- Two-factor authentication implemented from RFC 6238 with Node's crypto module (HMAC-SHA1, 30 s step, ±1 step tolerance, QR provisioning URL, 8 recovery codes). A password login for a 2FA account returns only a 10-minute temporary token until the code is verified;
- rate limiting with express-rate-limit: 100 requests per 15 min per IP across /api, and stricter limits for login and 2FA verification (10 per 15 min), registration (10 per hour), password reset and verification resend (5 per 15 min);
- helmet security headers, a 10 MB JSON body limit, and Cloudinary uploads restricted to image and video formats with a 50 MB cap; destructive admin actions require password re-entry.

User Interface, UX and Responsiveness

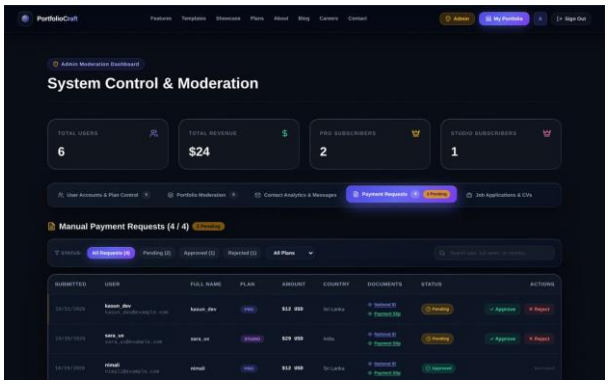
The interface uses a consistent dark, glass-style design system built with Tailwind CSS: shared button, input and card classes, one accent palette, and iconography from a single library. A fixed header with active-route highlighting provides navigation on every page and collapses to a drawer menu on small screens; a footer groups platform, company and legal links. Creators see ‘My Portfolio’ and ‘Sign Out’ actions and administrators an additional ‘Admin’ entry. Feedback is immediate: loading spinners, save and publish status badges, toast messages and confirmation modals. Figure 4 shows representative screens.



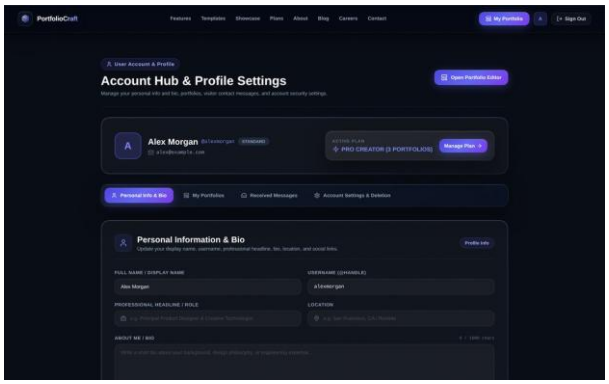
(a) Registration form (Create)



(b) Visual editor: sections, canvas, inspector (Update)



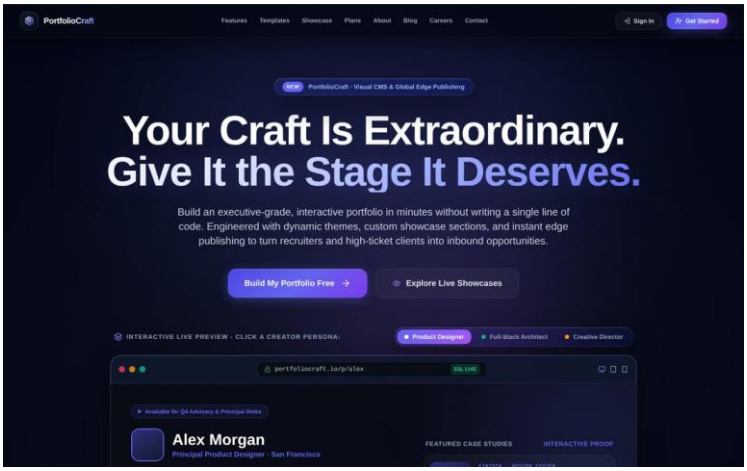
(c) Admin payment review with filters (Read / Update)



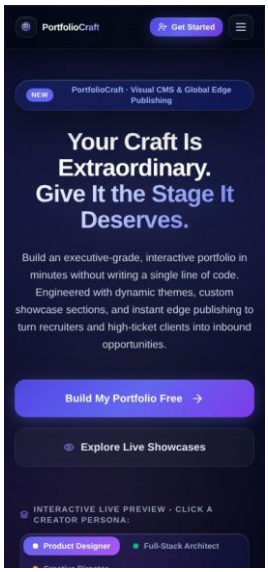
(d) Account hub: profile, portfolios, messages, deletion

Application screens captured from the running frontend (views (b) to (d) populated with sample data)

Responsiveness : The layout is mobile-first: the codebase uses Tailwind’s sm, md, lg and xl breakpoint prefixes in over 650 places, so grids collapse to a single column on phones. The editor adapts as well: on desktop it shows sections, canvas and inspector side by side; on narrow screens the side panels become toggled ‘Sections’ and ‘Customize’ panels. Inside the editor, a device switcher previews the portfolio at desktop, 768 px tablet and 390 px mobile widths, so creators can check their result without leaving the page.



Desktop (1440 px)



Mobile (390 px)

Code Organization and Structure

The repository separates concerns by folder. Backend code follows route : model layering with shared helpers for e-mail, TOTP and templates; the frontend isolates server access in one Axios instance with a token interceptor, keeps cross-page state in Redux slices, and reuses a single section registry for the editor canvas, the template preview and the public page. Roughly 22,000 lines of frontend and 4,200 lines of backend JavaScript make up the application.

Path	Contents
backend/server.js, config/	App setup (helmet, CORS, limiter, Passport), database, JWT strategy, Cloudinary
backend/routes/, models/, utils/	Eight route modules, six Mongoose models plus legacy Payment, e-mail and TOTP helpers; seed.js creates a sample admin and showcase portfolio
frontend/src/pages/, components/	27 pages; editor/ (sidebar, inspector), portfolio/ (12 section components plus the ComponentRenderer registry), common/ (header, footer)
frontend/src/store/, services/, data/	Redux slices, Axios instance, template and showcase data
.github/workflows/ci-cd.yml, documentation/	CI pipeline; seven Markdown guides (overview, CRUD, architecture, ERD, deployment, UI/UX, demo)

Naming is consistent (camelCase identifiers, PascalCase components, REST resource paths), and every route carries a @route / @desc / @access comment. Performance work includes route-level code splitting with React.lazy for the heavy Editor, Admin, Profile and Payment pages and Vite manual chunks for the React, state and UI vendor bundles. Version control is on GitHub; .gitignore keeps .env files and node_modules out of the repository, and a GitHub Actions workflow runs on each push and pull request to main, installing dependencies, syntax-checking the backend and building the frontend.

Installation, Configuration and Deployment

Prerequisites: Node.js 18 or later, npm, Git, and a MongoDB database (Atlas or local). Cloudinary and an e-mail provider are needed only for uploads and outgoing mail; without e-mail credentials the service logs the message instead of sending it.

```
git clone https://github.com/rsmpdo/portfolio-platform.git
cd portfolio-platform/backend && npm install
# create backend/.env (variables below), then:
node seed.js        # optional: sample admin and showcase portfolio
npm run dev         # API on http://localhost:5000  (npm start for production)

cd ../frontend && npm install
npm run dev         # Vite on http://localhost:5173, proxies /api to port 5000
npm run build       # production bundle in frontend/dist
```

Backend .env variable	Purpose
MONGODB_URI	MongoDB connection string
JWT_SECRET, JWT_EXPIRE	Token signing secret and lifetime (default 30d)
CLIENT_URL, ADMIN_SECRET_CODE	Public site URL used in e-mail links; code that grants the admin role at registration
CLOUDINARY_CLOUD_NAME, _API_KEY, _API_SECRET	Media storage
GMAIL_USER + GMAIL_APP_PASSWORD, or SENDGRID_API_KEY	Outgoing e-mail (Gmail SMTP first, SendGrid as fallback)

Deployment : The React build is hosted on Vercel, where frontend/vercel.json rewrites /api/ requests to the Render-hosted API and all other paths to index.html for client-side routing. The API runs as a Render web service defined in render.yaml (build npm install, start npm start, secrets supplied as environment variables), connected to MongoDB Atlas. The live site is available at https://portfoliocraftlk.vercel.app.

Limitations, Future Work and Conclusion

Reviewing the code against the rubric highlighted several improvements that would be made next:

- **Automated testing:** there are no unit or API tests yet; CI only builds the client and checks syntax. Jest with Supertest (API) and Vitest (client) would protect the CRUD rules.
- **Input validation:** checks are hand-written per route. A schema library (Joi or express-validator) would unify them, and PUT /layouts/me should pre-check handle uniqueness so a duplicate returns a clear 400 instead of a database error.
- **Secrets and CORS:** development fallbacks for the JWT secret and admin code and a permissive CORS fallback, should be replaced by mandatory environment variables and a strict origin list; 2FA recovery codes should be stored hashed.
- **Analytics and reporting:** the "unique visitors" counter currently increments on every view, and the admin revenue figure reads the legacy Payment collection rather than approved payment requests; both need a visitor key and a query on PaymentRequest.
- **Feature gaps:** drag-and-drop section ordering (currently move up/down), a newsletter unsubscribe route, and search and sort on the public Showcase gallery.

Conclusion : PortfolioCraft meets its objectives: it provides a working, deployed MERN application with persistent CRUD on its main Layout entity and five supporting collections, server-enforced business rules (plan limits, last-portfolio protection, ownership checks), layered security including two-factor authentication, a normalised and indexed database design, and a responsive visual editor with undo/redo and live device preview. The project strengthened practical skills in REST API design, document modelling, state management and cloud deployment, and the identified gaps provide a clear plan for further development.

Refer /documentation folder and readme.md file in GitHub repository for more details.

References

[1] React documentation, https://react.dev; Redux Toolkit, https://redux-toolkit.js.org; Tailwind CSS, https://tailwindcss.com/docs

[2] Express, https://expressjs.com; Mongoose, https://mongoosejs.com/docs; MongoDB Manual, https://www.mongodb.com/docs

[3] M'Raihi et al., "TOTP: Time-Based One-Time Password Algorithm", RFC 6238, https://datatracker.ietf.org/doc/html/rfc6238

[4] Cloudinary documentation, https://cloudinary.com/documentation; Vercel and Render deployment documentation